

ASSIGNMENT 1 FOR LEIGH

DAV24889747 (Cruz Joseph Davies)
GOWER COLLEGE SWANSEA [Company address]

Contents

Worksheet 1 - Pineapple Program	2
Description of the program.....	2
The User Interface.....	2
Interface Improvements	3
The Code	3
Code Improvements.....	4
Draw a flowchart for the logic (Zoom in).....	5
Worksheet 2 - Lunchtime Program	6
Description of program.....	6
The User Interface.....	6
Interface Improvements	6
The Code	7
Code Improvements.....	7
Worksheet 3 - Problem Definition Report	8
Business Requirements.....	8
Program Inputs	8
Program Outputs	8
Relationships between inputs and outputs	8
Worksheet 4 - Screen designs	9
Interface Designs (Zoom in).....	9
Worksheet 5 & 6 - Data Dictionary + Flowchart.....	11
Data Dictionary	11
Program Flowchart (Zoom in)	11
Worksheet 7 – Test Plan.....	13

Worksheet 1 - Pineapple Program

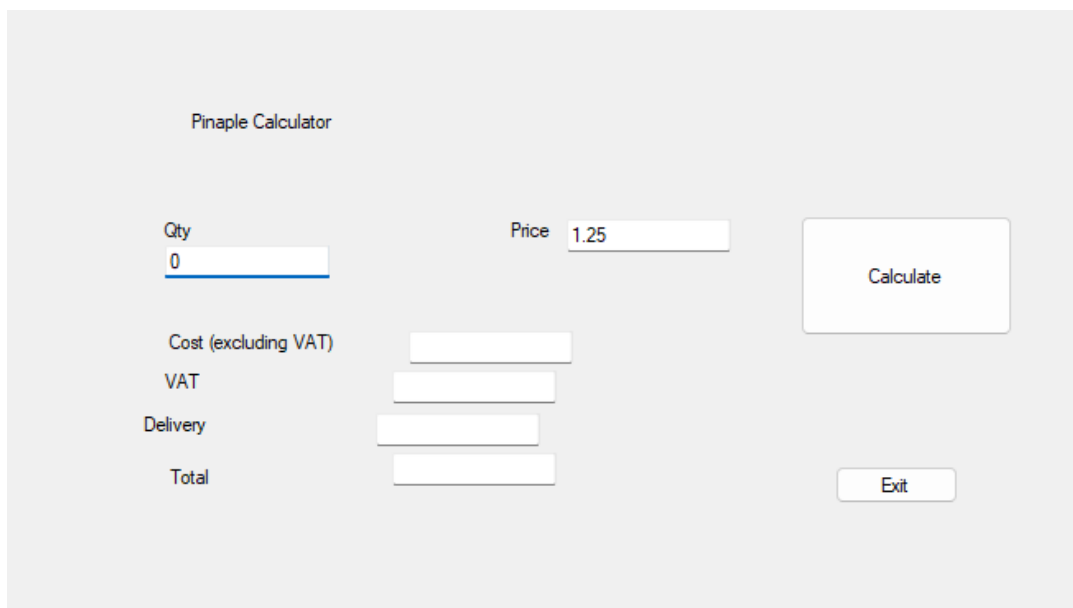
Description of the program

The Pineapple Calculator is a tool designed to make the process of calculating order totals simple. When the application starts it greets the user with a welcome message, giving a friendly tone for the start. The user then enters the quantity of items and the price per item into specific text boxes. By clicking the calculation button, the program automatically processes the data determining the total, then applying a 20% VAT and calculating a delivery fee based on how much the user has spent. This process is automatic so ensures that the user receives an accurate final total without needing to use math.

The User Interface

1. Describe the form. The text / colours / controls (lists/text boxes etc.) be specific

The form is a calculator and has a simple grey background. On the left it has labels for Quantity, Cost (excluding VAT) VAT, Delivery and Total. they each have a small white text box beside it for input or display. To the right of the quantity button is another text box labeled Price for the user to enter a value like 5. There's a Calculate button on the right and a smaller Exit button below it. The design is very simple using the default Windows fonts and simple white text boxes with black text.



The screenshot shows a window titled "Pinapple Calculator" (note the spelling). The interface is set against a light grey background. On the left side, there are five labels: "Qty", "Cost (excluding VAT)", "VAT", "Delivery", and "Total". Each label is followed by a white text box. The "Qty" box contains the number "0". To the right of these labels, there is a "Price" label followed by a white text box containing "1.25". Further to the right, there is a large white button labeled "Calculate". Below the "Calculate" button, there is a smaller white button labeled "Exit".

2. What is good – all the positive features of the program. Why are they good?

The program includes a few features that make it user friendly. When the program starts, the user is greeted with a welcome message box. The interface is relatively okay in terms of design because all of the labels on the form are clear and each of their purposes are simple, making the program easy to navigate. It's easy for the user to understand what inputs are required inside the textbox and what their outputs will be. Using dropdown menus for currency selection ensures the user can only pick from supported currencies. And the validation that checks for an amount is a great feature because it ensures accuracy and that the program will not take any the invalid data.

Interface Improvements

3. What can be improved – What is not good, and HOW can it be improved.

The UI here has many issues that need to be corrected to make it look more user friendly and professional. The labels and text boxes at the bottom are messy and should be aligned to become neat and even. There's also some spelling mistakes in the welcome message, such as "Helo" and "Pinaple," which need to be fixed. Finally, the output boxes currently let the user type anything in them, which could lead to errors, and makes the program look rushed, so these should be locked or changed to labels so they are display only.

The Code

4. List each event that is programmed. For each event, discuss the programming features

button1_Click (Calculate Button)

```
0 references
Private Sub button1_Click(sender As Object, e As EventArgs) Handles button1.Click
    Dim quantity, delivery As Integer
    quantity = 0
    delivery = 0
    Dim price, cost, VAT, total As Double
```

This event runs when you click the Calculate button. It reads the quantity and price from the text boxes, calculates the cost, VAT, and delivery charge then displays the total price. Finally it updates the output labels to show the user the final summary and the total price.

button2_Click (Exit Button)

```
Private Sub button2_Click(sender As Object, e As EventArgs) Handles button2.Click
    Dim DialogResult1 As DialogResult = MessageBox.Show("Are You sure you want to quite", "Pinaple Calculator", MessageBoxButtons.YesNo)

    If DialogResult1 = DialogResult.Yes Then
        Application.Exit()
    End If
```

This event runs when you press the exit button, it displays the confirmation message asking the user if they're sure they want to exit the program. This feature is useful because it provides a clear way for the user to end the program rather than just forcing the window shut.

form1_load (Entering Program)

```
0 references
Private Sub Form1_Load(sender As Object, e As EventArgs) Handles MyBase.Load
    MessageBox.Show("Helo and welcom to the Pinaple Calculator")
End Sub
```

This event runs as soon as you start the program, it displays the welcome message. Its main job and only job is to display the welcome message box that greets the user and identifies the program as the "Pineapple Calculator."

5. Did it use conditions – describe any IF statements – Why were they needed? What purpose did they serve?

```
If cost < 10 Then
    delivery = 5
ElseIf cost >= 10 And cost < 20 Then
    delivery = 2
ElseIf cost >= 20 Then
    delivery = 0
End If
```

Yes, the program does use conditions through IF statements, they're used inside of the button1_Click event in order for the program to decide the correct delivery charge based on the total order cost. So the code evaluates the total cost of the order to figure out the delivery fee, so if the cost is low like under £10 it applies a higher shipping charge but for larger orders over £20, it rewards the user with free delivery. Without this condition the program couldn't handle different pricing tiers and the delivery charge would remain static regardless of the order size being high or low. By using this structure the program can adjust the final total making the calculation process dynamic and accurate for every customer.

Code Improvements

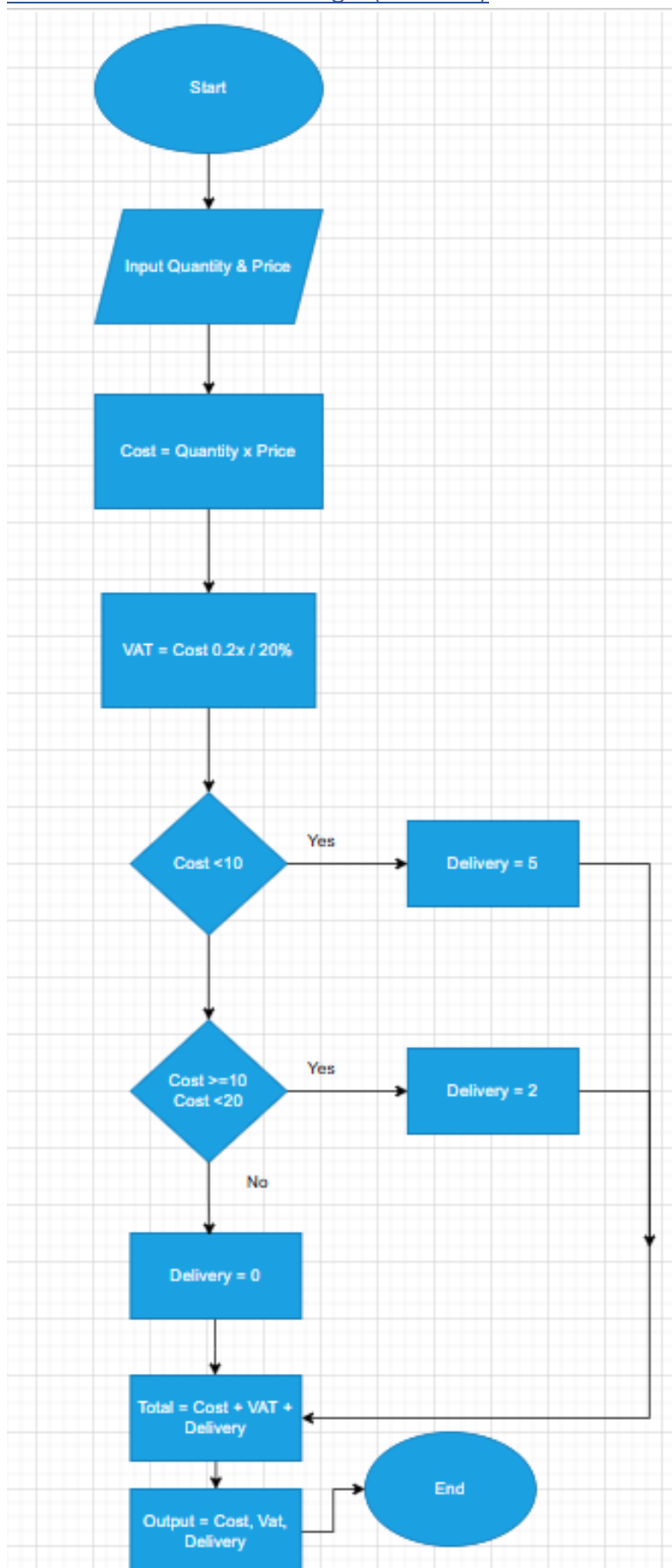
```

    If cost >= 10 & cost < 20 Then
        delivery = 2
    If cost >= 20 Then
        delivery = 0
        total = cost + VAT + delivery

        textBox3.Text = cost.ToString()
        textBox4.Text = VAT.ToString()
        textBox5.Text = delivery.ToString()
        textBox6.Text = total.ToString("£0.00")
    End If
End If
End If
End Sub
```

A big issue in the current code is the usage of 'stacked' If statements, which creates a conflict where some of the conditions are never reached. Because the program gets stuck in the first check, it'll fail to trigger the calculations needed for any higher costs, resulting in incorrect delivery cost and total fees. They could be replaced with a clear Elself structure would improve the codes efficiency and ensure that every cost range is processed accurately and properly.

Draw a flowchart for the logic (Zoom in)



Worksheet 2 - Lunchtime Program

Description of program

The program allows you to add a food option to a list box using the new menu item textbox, and clear the food from the textbox. After it's added to the list you can click on the food and it'll warn you with a label if it's a healthy food or not, when you click on a button you generate an event, as it is an event driven program

The User Interface

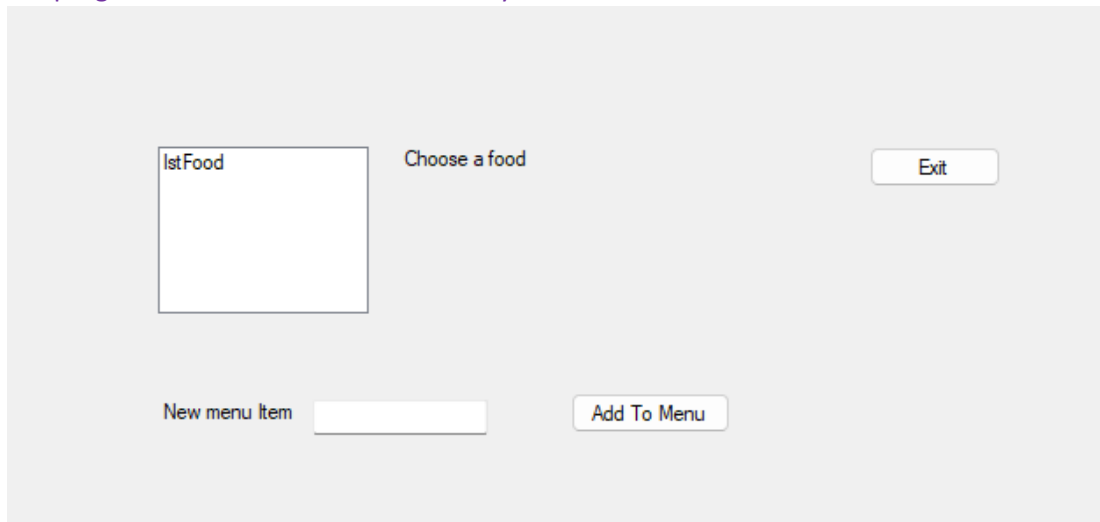
1. Describe the form. The text / colours / controls (lists/text boxes etc.) be specific.

The form uses a light grey background with simple black text. It features a white list box on the left containing the text "lstFood" and a white text box at the bottom for typing. There's two rectangle buttons and one is labelled "Exit" at the top right and one is labelled "Add To Menu" at the bottom. Labels like "Choose a food" and "New menu item" are placed next to these controls to guide the user.

Interface Improvements

2. What can be improved – What is not good, and HOW can it be improved

The UI interface could be improved by aligning the controls in a straight line, as right now they're all spread out and look unorganised. This would make the form clearer and easier to use. The list name "lstFood" should be changed to a more user friendly label such as "Food Menu" so it does not look like code. The "Add To Menu" button could be made a brighter colour, such as green so it stands out and is easier to notice. Also the text box should automatically clear after an item is added to make the program faster and more user-friendly.



The Code

3. List each event that is programmed. For each event, discuss the programming features

This code uses a btnAdd_Click event that runs when the user clicks the Add button. It uses an IF statement to check whether the txtNewFood text box is empty, which is a form of input validation. This stops the user from adding a blank item to the list. And, if the text box is empty then a message box gets displayed to tell the user to enter the name of a new food and the Return statement stops the rest of the code from running. If text is entered then food item is added to the list and then the text box is cleared and focus is returned to the text box to improve usability.

```
Private Sub btnAdd_Click(sender As Object, e As EventArgs) Handles btnAdd.Click
    If txtNewFood.Text = "" Then
        MessageBox.Show("Please enter the name of the new food to be added")
        Return
    End If
```

The btnExit_Click event uses the Application.Exit() command to close the program. It allows the user to safely exit the application.

```
Private Sub btnExit_Click(sender As Object, e As EventArgs) Handles btnExit.Click
    Application.Exit()
End Sub
```

The lstFood_SelectedIndexChanged event happens when the user selects a food item from the list box. This event uses logic to decide what message to display to the user. If the selected item is “Chips” or “Burger”, the label shows a warning that it is an unhealthy option. For any other food item a positive message is displayed. This allows the program to respond differently depending on your selection.

```
Private Sub lstFood_SelectedIndexChanged(sender As Object, e As EventArgs) Handles lstFood.SelectedIndexChanged
    If lstFood.Text = "Chips" Or lstFood.Text = "Burger" Then
        lblInformation.Text = "unhealthy Option"
    Else
        lblInformation.Text = "yummy - that sounds good"
```

Code Improvements

4. Can you think of any improvements to the code?

An improvement to this code would be to add a “Clear All” button that removes all the items from the food list, allowing the user to reset the list without restarting the application. Another improvement would be to make the food checks “case insensitive” because the program currently only recognises the exact words “Chips” or “Burger”. So this means using lowercase letters won’t trigger the warning message. Converting the text to a standard format before checking would improve the user experience.

Worksheet 3 - Problem Definition Report

Business Requirements

The context I have been given is that I'm being asked to create and design a currency converter that allows the user to convert one currency to another, so my program should take the input from the textbox and then convert the inputted currency and then show what the result is. It should support a sensible amount of min and max converted currency and an attempt to input an amount outside of that should display an error message telling the user to select an amount between 1 and 100000

My currency converter solution should allow multiple currencies to be inputted (GBP, USD, Euros, etc)

Program Inputs

The program should take any user entered amounts that are placed in a textbox, the amount must be a numeric value and should stay within the minimum and maximum range of 1 and 100000 given. text should be disabled or display an error message. If not within the set range then the program should display an error message telling the user to input a value in the given range.

The currency should be chosen from a dropdown menu that allows the user to select from a variety of different currencies (GBP, USD, Euros etc)

Program Outputs

The program outputs are designed to provide feedback that changes based on the user's interaction with the program. When the user meets all the validation criteria then the primary output is the calculated conversion value which is assigned to the results label to show the user the exact exchange amount. but if the input data fails any checks the output shifts to error messages delivered via pop up boxes, such as alerts for empty boxes or absurd values, so identical currency selections or values that fall outside the set range. Also the program also generates output things including a welcome greeting during the start up form load and a polite thank you message dialogue when the user presses the exit button to close the application.

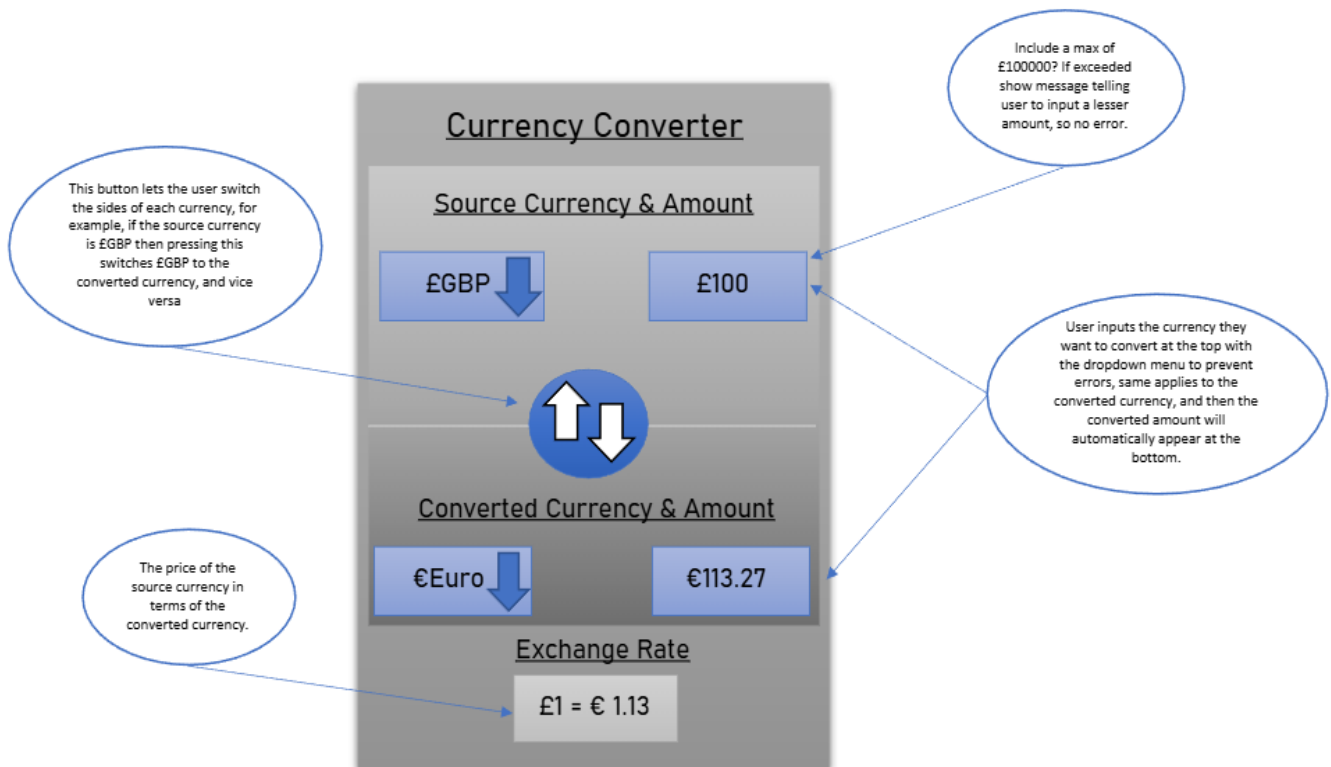
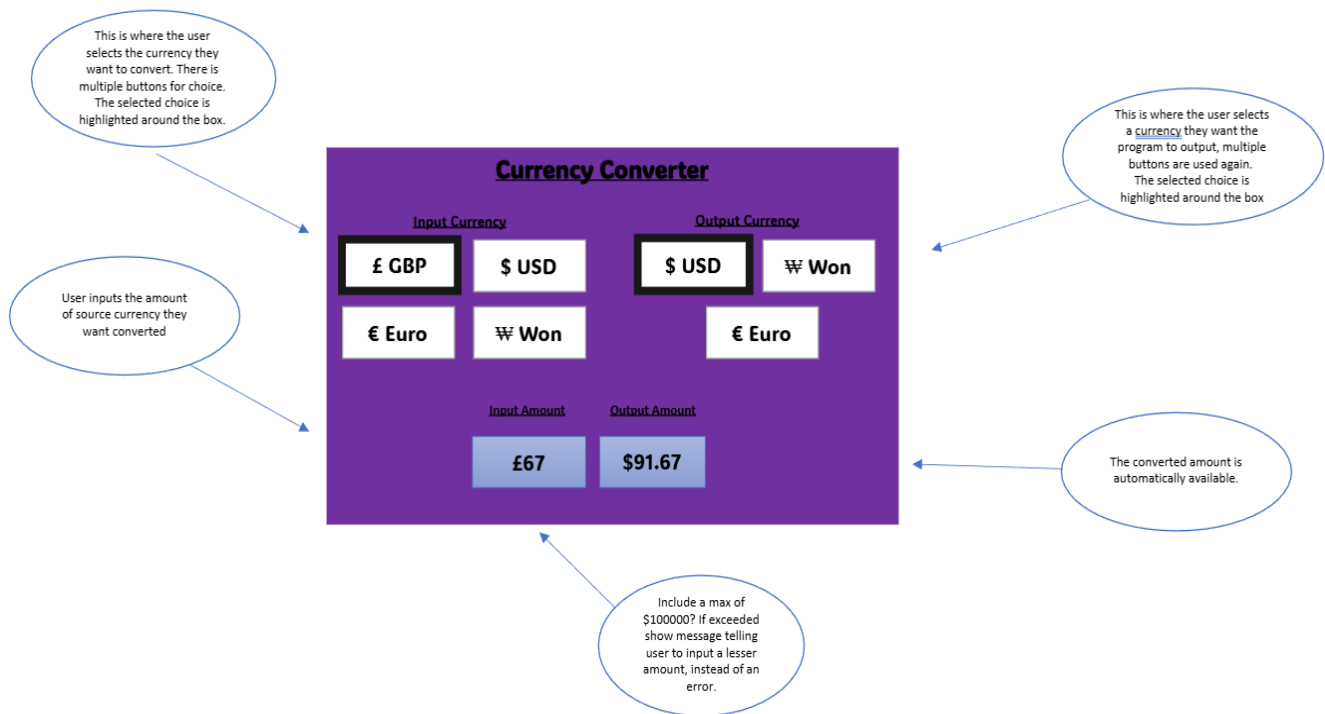
Relationships between inputs and outputs

The output the program generates is linked to what the user inputs. If the user enters a valid number within the allowed range and selects a currency like GBP or USD then output will be the calculated conversion. But if the user types an amount that isn't valid (such as a number higher than 10,000 or a negative number) then output will switch from a calculation to an error message.

If the user enters an amount less than £30, the program will still process the conversion as long as the amount is between 1 and 100000, if the user selects an amount outside of this range, so 100001 or 0 then the conversion will not be processed and the program will instead display an error message.

Worksheet 4 - Screen designs

Interface Designs (Zoom in)

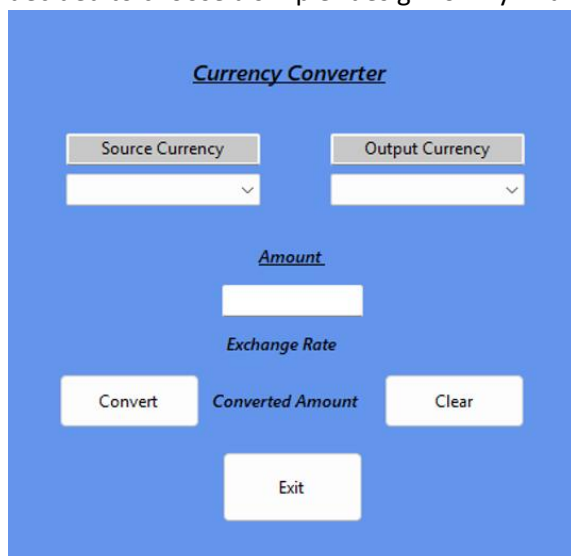


My first draft design has a purple interface where the users select their input and output currencies using a pair of buttons. To improve the programs navigation, I made it that the selected currency is highlighted with a black border when clicked. This design has a maximum input limit of 100000 so if the user tries to enter a higher amount then the program displays the message box instead of crashing with an error.

My second design was made with a more complex grey interface that uses dropdown menus for currency selection to save space compared to my first. A unique feature of this layout is the Switch button that allows you to instantly swap the source and converted currencies, also this version displays the live exchange rate at the bottom and automatically updates the converted amount as the user types, while keeping the same 100000 limit as the first design.

Explanation as to why I rejected my alternative:

I rejected these because they didn't have the proper balance by being simple and functional. I turned down the first one because having separate buttons for every currency takes up too much space on the screen, so in the future if I was to add more currencies then this design would make the interface look cluttered and difficult to navigate. I rejected the second design because features like the Switch button and the live exchange rate display add extra complexity that some people might find confusing, and because my goal was to create a tool that is simple, that anyone can use, I decided to choose a simpler design for my final one for the best balance.



The screenshot shows a 'Currency Converter' application with a blue background. At the top, the title 'Currency Converter' is underlined. Below it are two dropdown menus labeled 'Source Currency' and 'Output Currency'. In the center is a text input field labeled 'Amount'. Below the input field is the label 'Exchange Rate'. At the bottom, there are four buttons: 'Convert', 'Converted Amount', 'Clear', and 'Exit'.

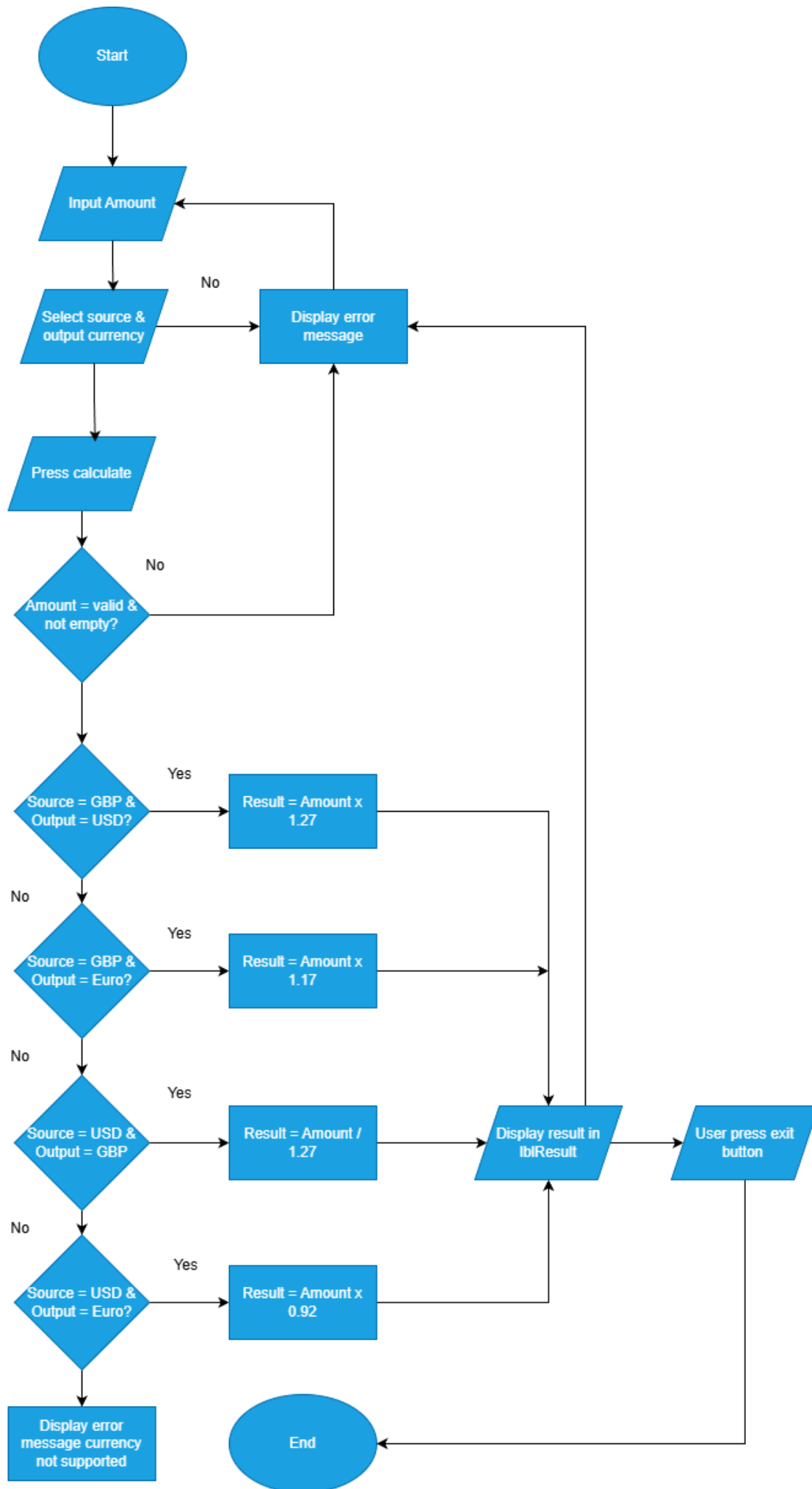
My final layout has a blue theme that keeps everything in one column. I decided to use dropdown menus for selecting both the source and output currencies. This is a key design decision because it prevents users from making typing errors when choosing a currency which makes the program more reliable. It also keeps the screen from getting cluttered with buttons, making it very easy for someone to understand.

Worksheet 5 & 6 - Data Dictionary + Flowchart

Data Dictionary

Data Name	Data Type	Where used	Input/Output/Internal	Description
amount	Double(variable)	btnConvert_Click	Internal	Stores the number value retrieved from the textbox to be used for the final calculation.
result	Double(variable)	btnConvert_Click	Internal	Stores the final calculated value after the exchange rate has been applied.
txtAmount.Text	String	btnConvert_Click	Input	The text entered by the user which is the amount of money they want to convert.
CbSource.Text	String	btnConvert_Click	Input	The currency type selected by the user to convert <i>from</i> (like GBP)
CbOutput.Text	String	btnConvert_Click	Input	The currency type selected by the user to convert <i>to</i> (like USD)
lblResult.Text	String	btnConvert_Click	Output	Displays the final converted amount on the converter for the user to see.

Program Flowchart (Zoom in)



Worksheet 7 – Test Plan

Test Number	Description	Excepted Results	Actual Results	Corrective action
1	Welcome message appears when program runs	Message box loads		
2	Combo boxes items populate when form loads	Items populated		
3	100 GBP to USD conversion	127USD		
4	User enters non-numeric values into the conversion box	Message Box – warning user to enter a sensible value		
5	User enters nothing into the conversion box	Message Box – warning user to enter a sensible value		
6	User exits the program	A confirmation message appears for the user		
7	When converting between	USD to GBP would equal a conversion rate of 0.79		

	different currencies			
8	When converting between different currencies	USD to Euros would equal a conversion rate of 0.92		
9	When converting between the same currency	Message Box – warning user to select two different currencies		
10	User selects an extremely high number	Message Box – warning user to select a reasonable number		
11	User selects an extremely low number	Message Box – warning user to select a higher number		
12	User types letters and numbers together instead of numbers only	Message Box – warning user to insert numeric values instead of letters		